Studio Nx

백엔드 234 — 악기 칸

2026-10-18

Backend(FastAPI/Python) 234 — `track_service` 악기 칸 (칸 번호 확인과 네 함수)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 악기 칸 읽기·붙이기·고치기·떼기 (PRD 02 FR-245, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	233 트랙과 권한 · 235 이펙트 칸

1. 하는 일

트랙 안의 악기 칸(`track_instruments`)을 다루는 함수 다섯 개를 둡니다.

함수	하는 일	권한
`_require_free_slot`	같은 트랙의 같은 칸에 이미 행이 있는지 확인하고, 있으면 400	쓰기 준비
`list_instruments`	트랙의 악기 칸 목록 (칸 번호 순)	멤버면 누구나
`add_instrument`	악기 한 칸 붙이기. `INSTRUMENT` 플러그인만 가능	OWNER, EDITOR
`update_instrument`	파라미터, 볼륨, 팬, 솔로, 켜기, 칸 번호 고치기	OWNER, EDITOR
`remove_instrument`	악기 칸 떼기	OWNER, EDITOR

악기 칸은 **가상악기(INSTRUMENT)만** 들어갑니다. 이펙트를 넣으려 하면 400 입니다. 종류를 섞으면 소리 처리 순서가 꼬이기 때문입니다.

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/instruments`, `PATCH/DELETE /tracks/{id}/instruments/{iid}` (239)	네 라우터가 이 함수들을 부릅니다
화면의 [Instrument] 칸	악기를 붙이고 파라미터를 저장할 때 부릅니다 (FR-262·264)
S1-10 저장·불러오기	트랙의 악기 구성을 함께 저장하고 되살립니다 (FR-254)

3. 코드

칸 번호 확인 (S1-09 에 추가한 안전장치)

```
def _require_free_slot(
    db: Session, model, track_id: int, slot_index: int, what: str, exclude_id: int | None = None
) -> None:
    """같은 트랙의 같은 칸에는 한 행만 넣을 수 있다 (DB UNIQUE (track_id, slot_index)).
```

```
    안 막으면 DB 가 IntegrityError 를 뱉어 500 이 된다. 화면이 이미 찬 칸에 넣으려 하면
    400 으로 알려 주어, 다른 칸 번호를 골라 다시 보내게 한다 (FR-263 순서 편집).
    exclude_id 는 고쳐지는 칸 자신이다 (자기 자리에 다시 넣는 것은 막지 않는다).
    """
    pk_name = model.__mapper__.primary_key[0].name
    stmt = (
        select(func.count())
        .select_from(model)
        .where(model.track_id == track_id, model.slot_index == slot_index)
    )
    if exclude_id is not None:
        stmt = stmt.where(getattr(model, pk_name) != exclude_id)
    if db.scalar(stmt):
        raise AppError(
            ErrorCode.BAD_REQUEST, 400, f"Slot {slot_index} is already used by another {what}."
        )
```

목록과 한 칸 찾기

```
def list_instruments(db: Session, user: User, track_id: int) -> Sequence[TrackInstrument]:
    track = require_track(db, user, track_id)
    stmt = (
        select(TrackInstrument)
        .where(TrackInstrument.track_id == track.track_id)
        .order_by(TrackInstrument.slot_index, TrackInstrument.track_instrument_id)
    )
    return db.execute(stmt).scalars().all()


def _require_instrument(db: Session, user: User, track_id: int, instrument_id: int):
    inst = db.get(TrackInstrument, instrument_id)
    if inst is None or inst.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track instrument not found.")
    _require_editor_track(db, user, track_id)
    return inst
```

붙이기 · 고치기 · 떼기

```
def add_instrument(db, user, track_id, *, plugin_id, slot_index=0, params=None, ...):
    _require_editor_track(db, user, track_id)
    plugin = plugin_service.require_plugin(db, plugin_id)
    if plugin.plugin_type != PluginType.INSTRUMENT:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Only INSTRUMENT plugins can be added to instrument
s.")
    _require_free_slot(db, TrackInstrument, track_id, slot_index, "instrument")
    inst = TrackInstrument(track_id=track_id, plugin_id=plugin_id, slot_index=slot_index,
                           params=params or {}, volume_db=volume_db, pan=pan,
                           is_solo=is_solo, is_enabled=is_enabled)
    db.add(inst)
    db.commit()
    db.refresh(inst)
    return inst


def update_instrument(db, user, track_id, instrument_id, **kwargs):
    inst = _require_instrument(db, user, track_id, instrument_id)
```

```
    if "slot_index" in kwargs and kwargs["slot_index"] is not None:
        _require_free_slot(db, TrackInstrument, track_id, kwargs["slot_index"], "instrument",
                           exclude_id=inst.track_instrument_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(inst, key):
            setattr(inst, key, value)
    db.commit()
    db.refresh(inst)
    return inst
```

문법 풀이

식	뜻
`model.__mapper__.primary_key[0].name`	그 모델의 기본 키 칸 이름을 꺼냅니다. 악기(`track_instrument_id`)와 이펙트(`track_plugin_id`)가 달라 이름을 받아 씁니다
`getattr(model, pk_name) != exclude_id`	"나 자신은 빼고" 세는 조건입니다
`db.scalar(select(func.count())...)`	개수를 하나의 값으로 받습니다. 0 이면 비어 있습니다
`f"Slot {slot_index} is already used..."`	오류 메시지에 칸 번호를 넣습니다. 화면이 어느 칸이 문제인지 알 수 있습니다
`.order_by(slot_index, track_instrument_id)`	칸 번호 순으로, 같으면 번호 순으로 정렬합니다
`params or {}`	안 보내면 빈 사전을 넣습니다. 악기의 파라미터 칸은 NULL 도 되지만, 화면이 바로 쓸 수 있게 빈 사전으로 둡니다
`inst.track_id != track_id`	경로의 트랙과 다른 트랙의 악기는 404 입니다. 주소를 손으로 바꿔 남의 악기를 고치는 일을 막습니다

4. 왜 이렇게

선택	이유
목록을 관계 대신 쿼리로	같은 요청 안에서 지우고 바로 다시 읽을 때, 관계(`track.instruments`)에는 지워진 행이 남아 있습니다(DB 삭제를 CASCADE 에 맡겨서). 쿼리로 읽으면 늘 실제 DB 값입니다
칸 번호를 미리 확인	DB 에는 `UNIQUE (track_id, slot_index)` 가 있어 위반하면 `IntegrityError`(500)가 납니다. 400 으로 알려 주면 화면이 다음 칸 번호를 골라 다시 보낼 수 있습니다 (FR-263)
`exclude_id` 로 자기 자리 허용	"볼륨만 바꾸기" 처럼 같은 칸을 그대로 다시 보내는 요청이 400 이 되면 안 됩니다
붙이기에서 플러그인 종류 검사	악기 자리에 이펙트가 들어가면 소리 처리가 뒤섞입니다. 400 으로 막습니다 (FR-245)
권한 검사를 `_require_editor_track` 한 곳으로	경로마다 같은 규칙을 다시 쓰면 한 곳만 빠뜨려도 구멍이 납니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, 이펙트를 악기 자리에 넣으면 400, 없는 플러그인이면 404
- 목록이 칸 번호 순이고, 지운 뒤에는 빈 목록
- 고치기(볼륨, 켜기)와 떼기 204
- 같은 칸에 두 번 넣으면 400, 옮길 칸이 이미 차 있으면 400, 자기 자리로는 200
- 로그인하지 않으면 401